

University of Tehran
College of Engineering
School of Electrical and Computer
Engineering



Distributed Systems

HW1

Full Name
Sepanta Ghonoodi
Sajad Hasanpour

Student ID
810102483
810102432

May 14, 2026

Table of Contents

Part 1	2
Architecture and Design Overview	2
IPC Mechanism & Auto-Initialization	2
Communication Protocol & Advanced Serialization	2
Error Handling and Resilience	2
Part 2	3
CPU-Bound task:	3
Mutex Task	4
Sleep Task	5
Table of Results	5
Final Conclusions:	8
Part 3	9
Overview & Environment	9
Network Setup: Port Forwarding	10
Execution Instructions	10
Build the Docker Image:	10
Run the Container:	10
Observability: Logging Middleware	10
Proof of Connectivity	10

Part 1

Architecture and Design Overview

In this section, we designed and implemented a robust Inter-Process Communication (IPC) system utilizing two independent Go processes: the Interface and the Worker. The architecture strictly follows a decoupled Request-Response model. The Interface acts as a lightweight, user-facing client responsible for capturing standard input and displaying the final results. Conversely, the Worker functions as a resilient background daemon. It continuously listens for incoming byte streams, performs floating-point mathematical operations, and returns structured data without terminating after a single execution.

IPC Mechanism & Auto-Initialization

The core communication bridge was established using Linux Named Pipes (FIFOs). Instead of relying on manual user setup via terminal commands, we engineered the system to be entirely self-contained by interacting directly with the Linux kernel. Upon initialization, the Worker process utilizes Go's `syscall.Mkfifo` to automatically generate the `req_pipe` and `res_pipe` files with `0666` (read/write) permissions.

To manage process synchronization efficiently, we leveraged the native blocking behavior of POSIX pipes. By opening the pipes with `os.O_RDONLY`, the Worker naturally suspends its execution (blocks) at the operating system level until the Interface opens the opposite end of the pipe for writing. This eliminates the need for aggressive CPU polling and ensures optimal resource management.

Communication Protocol & Advanced Serialization

The communication protocol was explicitly defined to maintain consistency and fault tolerance across process boundaries:

Request Format: The Interface parses user input and sends raw text streams in a strict `OP A B` format (e.g., `ADD 5.5 4.5`). The Worker intercepts these streams using a buffered I/O scanner (`bufio.NewScanner`) to safely process data line-by-line.

Response Format: To ensure a highly structured and scalable design, the Worker serializes the computational results into a JSON payload before returning them through the response pipe.

A significant architectural decision in our implementation was the use of **pointer-based JSON marshalling** (`*float64`). In Go, unassigned numerical values default to zero. By assigning a pointer to the result field combined with the `omitempty` struct tag, our system intelligently differentiates between an actual calculated zero (e.g., `SUB 5 5`), which is successfully serialized as `{"status": "OK", "result": 0}`, and an error state, where the result field is entirely omitted from the JSON payload.

Error Handling and Resilience

The system is fortified with comprehensive error handling to prevent unexpected panics and crashes. It natively intercepts computational violations (e.g., Division by Zero), structural inconsistencies (insufficient arguments), and type-casting failures (non-numeric string inputs), returning specific JSON error messages for each scenario. Furthermore, to prevent File Descriptor (FD) leaks during unexpected disconnects, Go's `defer` statements were strategically implemented to guarantee that all active pipes are safely closed before the function returns.

Part 2

CPU-Bound task:

In this section, we analyze a purely CPU-bound task that calculates a mathematical sum. The total computational workload remains constant across all tests, but it is divided into equal sub-parts and distributed among the active goroutines. Below, we present the total execution time, throughput, and average latency for each configuration.

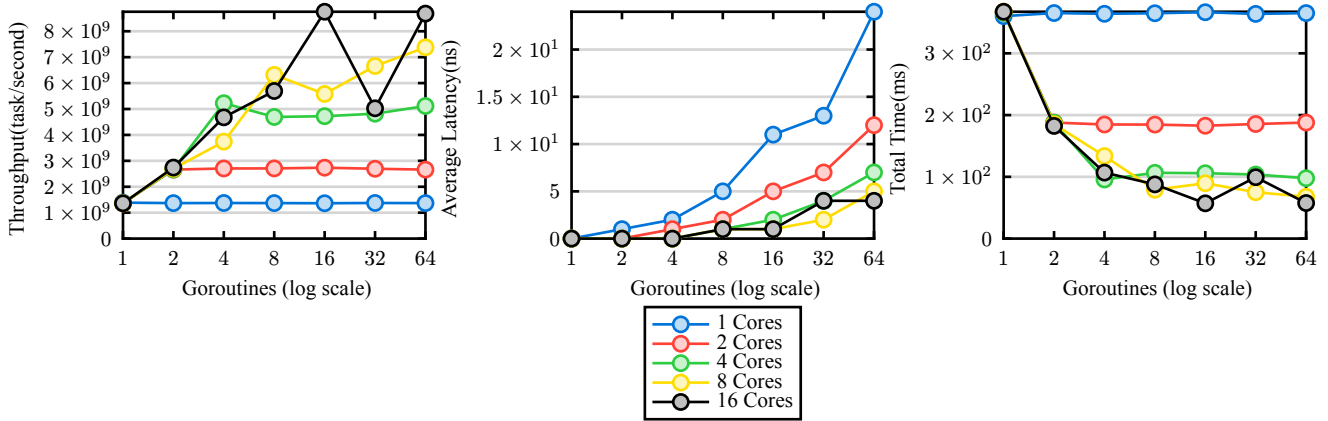


Figure 1: Performance benchmarks for CPU-bound tasks

This workload involves neither I/O waits (`sleep`) nor synchronization blocks (`mutex`). Consequently, when we increase the number of goroutines on a single physical core (`GOMAXPROCS=1`), we observe no improvement in throughput or total execution time. Instead, the average latency per task increases drastically. Because all goroutines are initialized simultaneously but forced to time-slice on a single core, they spend the majority of their lifecycle waiting in the scheduler's queue. While the absolute total execution time remains flat, the active lifespan of each individual worker is stretched, negatively impacting the average latency metric.

Furthermore, we observe that increasing the number of physical cores (`GOMAXPROCS > 1`) yields no performance boost if the number of goroutines remains lower than the available cores (e.g., 4 cores but 1 goroutine). In this state, the workload is not fully parallelized, leaving the additional physical CPU cores idle.

However, as the plots demonstrate, when multiple cores are available, increasing the number of goroutines creates a massive performance boost in both total time and throughput. This scaling continues optimally until the number of goroutines matches the number of physical cores. Beyond this peak—when goroutines exceed the available physical cores—we no longer see improvements. This proves that for purely computational workloads, adding more goroutines does not equate to better parallelization once hardware resources are fully saturated; it only reintroduces context-switching overhead.

Finally, we can see that even under optimal parallelization (adding both cores and goroutines), the average latency still trends upward. This occurs because parallelization improves overall throughput, but it does not reduce the execution time of an individual worker's batch. Since each worker measures its latency from the start of the application to its respective finish line, adding more concurrent workers inherently increases the average measured turnaround time of the tasks.

Mutex Task

In this section, we examine a task constrained by a Mutex, where multiple goroutines attempt to increment a shared variable within a critical section.

When limited to a single physical core (GOMAXPROCS=1), increasing the number of goroutines does not trigger a catastrophic drop in performance. Because only one operating system thread executes at any given microsecond, the lock is acquired and released sequentially. Consequently, there is no true hardware-level contention for the memory address, keeping the throughput relatively stable.

However, when we increase the number of physical cores alongside the number of goroutines, we observe a severe degradation in performance. In a multi-core environment, parallel CPU cores actively compete to acquire the exact same lock to mutate the shared variable. This introduces massive hardware-level contention, as the cores constantly invalidate each other's caches.

Furthermore, this hardware contention compounds with software-level overhead. As multiple goroutines fail to acquire the lock, the Go scheduler is forced to spend a significant portion of its total execution time managing the traffic—spinning, parking (putting to sleep), and subsequently waking up blocked goroutines.

Therefore, for highly contended workloads, throwing more hardware resources (cores) and software concurrency (goroutines) at the problem does not improve throughput. Instead, the compounding overhead of cache contention and scheduler thrashing leads to a drastic loss of overall performance.

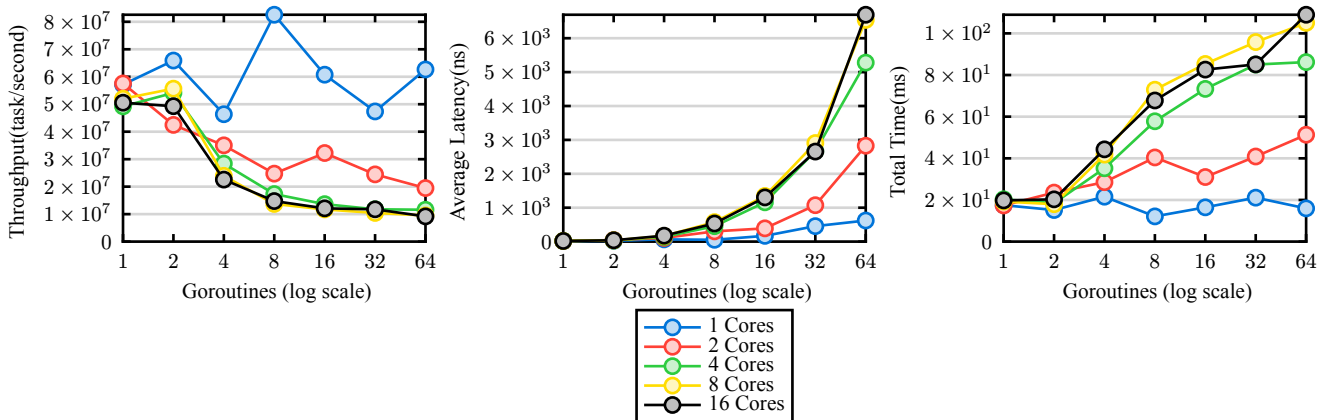


Figure 2: Performance benchmarks for Mutex-bound tasks

Sleep Task

In this final section, we introduce a time.Sleep element to the workload, simulating an I/O-bound task such as waiting for a network response or a database query.

When executing this workload with only a single goroutine, performance is exceptionally poor. Because the execution is entirely sequential, the single worker must initiate a task, enter a sleep state, and wait for the entire duration to pass before moving on to the next operation. The CPU remains completely idle while the thread is blocked, resulting in the longest total execution time and the lowest possible throughput.

However, by increasing the number of goroutines, we observe a massive improvement in performance, even when constrained to a single physical core (GOMAXPROCS=1). This demonstrates the core advantage of Go's concurrency model for I/O-bound tasks: latency hiding. When a goroutine enters a sleep state, it voluntarily yields the CPU. The Go scheduler immediately context-switches, placing a different, ready goroutine onto the core. By rapidly interleaving thousands of sleeping tasks, the scheduler ensures the physical CPU is never left waiting idly.

As demonstrated by the data, assigning more physical cores (e.g., GOMAXPROCS=16) to this highly concurrent workload consistently improves performance. Because the computational work executed between sleep cycles is microscopically small, having additional physical cores allows the Go scheduler to process the rapid waking and sleeping of thousands of goroutines in true parallel. This prevents a single OS thread from becoming a bottleneck when juggling thousands of context switches.

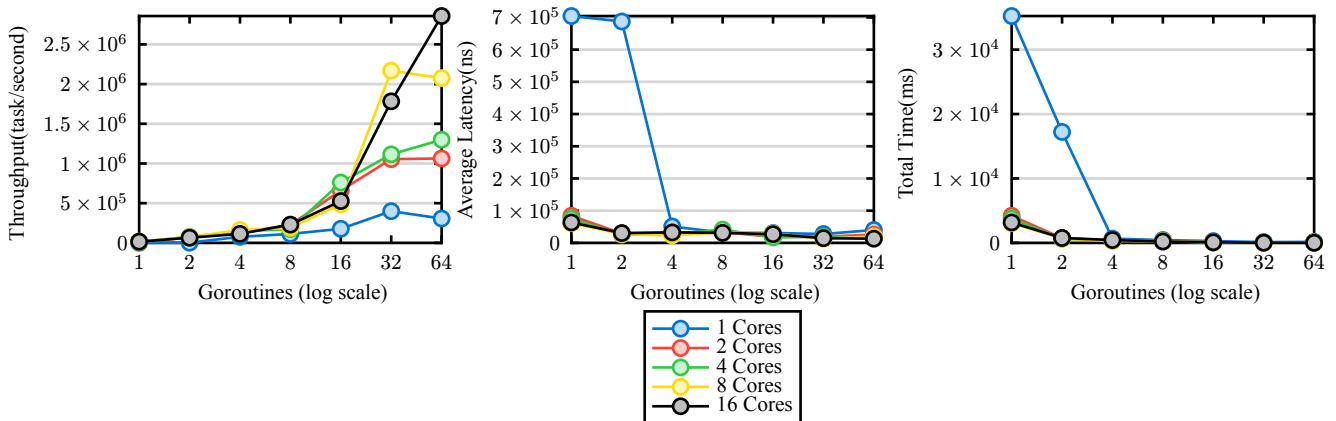


Figure 3: Performance benchmarks for Sleep-bound tasks

Table of Results

Here is the raw results of the code in the format of a table for each configuration.

Workload	GOMAXPROCS	Goroutines	TotalTime_ms	AvgLatency_ns	Throughput
CPU-bound	1	1	360.188	0	1388165878.12
CPU-bound	1	2	365.196	1	1369126284.97
CPU-bound	1	4	363.977	2	1373713834.67
CPU-bound	1	8	364.858	5	1370396788.88
CPU-bound	1	16	366.518	11	1364189941.99
CPU-bound	1	32	363.799	13	1374387006.22
CPU-bound	1	64	365.124	24	1369397444.83

CPU-bound	2	1	366.751	0	1363321591.27
CPU-bound	2	2	187.874	0	2661354064.59
CPU-bound	2	4	184.746	1	2706417780.45
CPU-bound	2	8	184.625	2	2708186120.85
CPU-bound	2	16	182.746	5	2736041381.73
CPU-bound	2	32	185.543	7	2694795938.36
CPU-bound	2	64	187.871	12	2661398899.50
CPU-bound	4	1	365.680	0	1367317132.88
CPU-bound	4	2	187.476	0	2667005895.59
CPU-bound	4	4	95.700	0	5224675901.86
CPU-bound	4	8	106.535	1	4693298394.17
CPU-bound	4	16	105.875	2	4722550355.52
CPU-bound	4	32	103.750	4	4819265031.24
CPU-bound	4	64	97.745	7	5115336724.82
CPU-bound	8	1	367.296	0	1361300259.77
CPU-bound	8	2	185.230	0	2699346204.31
CPU-bound	8	4	133.651	0	3741079330.87
CPU-bound	8	8	79.143	1	6317713250.62
CPU-bound	8	16	89.609	1	5579826810.21
CPU-bound	8	32	75.091	2	6658613827.88
CPU-bound	8	64	67.751	5	7379924350.46
CPU-bound	16	1	367.302	0	1361275835.85
CPU-bound	16	2	182.421	0	2740907486.18
CPU-bound	16	4	106.888	0	4677786226.07
CPU-bound	16	8	87.799	1	5694801228.49
CPU-bound	16	16	57.126	1	8752560255.16
CPU-bound	16	32	99.452	4	5027526663.71
CPU-bound	16	64	57.599	4	8680720579.67
Mutex-bound	1	1	17.499	17	57146857.42
Mutex-bound	1	2	15.174	24	65900767.94
Mutex-bound	1	4	21.570	66	46360950.51
Mutex-bound	1	8	12.112	54	82564472.33
Mutex-bound	1	16	16.461	170	60748060.12
Mutex-bound	1	32	21.108	456	47374767.53
Mutex-bound	1	64	15.960	622	62657332.56
Mutex-bound	2	1	17.371	17	57565940.49
Mutex-bound	2	2	23.541	45	42478934.70
Mutex-bound	2	4	28.540	109	35038584.14
Mutex-bound	2	8	40.423	302	24738532.88

Mutex-bound	2	16	31.017	391	32240324.02
Mutex-bound	2	32	40.870	1074	24467723.63
Mutex-bound	2	64	51.320	2828	19485512.33
Mutex-bound	4	1	20.318	20	49217965.90
Mutex-bound	4	2	18.474	36	54130720.71
Mutex-bound	4	4	35.225	139	28388526.16
Mutex-bound	4	8	57.790	456	17304070.47
Mutex-bound	4	16	73.338	1159	13635488.30
Mutex-bound	4	32	85.034	2669	11759963.16
Mutex-bound	4	64	86.202	5280	11600713.15
Mutex-bound	8	1	19.232	19	51996188.26
Mutex-bound	8	2	17.965	35	55664348.43
Mutex-bound	8	4	41.783	165	23933352.12
Mutex-bound	8	8	72.951	575	13707832.79
Mutex-bound	8	16	85.313	1346	11721481.33
Mutex-bound	8	32	95.824	2908	10435821.18
Mutex-bound	8	64	105.051	6539	9519216.00
Mutex-bound	16	1	19.779	19	50557742.91
Mutex-bound	16	2	20.294	39	49276859.62
Mutex-bound	16	4	44.303	175	22572001.98
Mutex-bound	16	8	67.772	534	14755368.60
Mutex-bound	16	16	82.604	1304	12106013.28
Mutex-bound	16	32	85.093	2656	11751788.73
Mutex-bound	16	64	108.945	6692	9178954.79
Sleep-bound	1	1	35218.428	704368	1419.71
Sleep-bound	1	2	17224.966	687387	2902.76
Sleep-bound	1	4	654.046	51057	76447.27
Sleep-bound	1	8	442.726	31921	112936.61
Sleep-bound	1	16	281.976	30906	177319.84
Sleep-bound	1	32	125.176	27598	399436.00
Sleep-bound	1	64	163.075	39923	306606.63
Sleep-bound	2	1	4211.379	84227	11872.60
Sleep-bound	2	2	736.457	29407	67892.59
Sleep-bound	2	4	401.356	31988	124577.76
Sleep-bound	2	8	219.833	33399	227445.35
Sleep-bound	2	16	75.858	18697	659127.74
Sleep-bound	2	32	47.396	19523	1054935.05
Sleep-bound	2	64	47.028	25709	1063205.87
Sleep-bound	4	1	3766.420	75328	13275.21

Sleep-bound	4	2	649.298	25930	77006.26
Sleep-bound	4	4	341.092	26425	146588.01
Sleep-bound	4	8	288.209	41399	173485.50
Sleep-bound	4	16	65.728	17006	760714.86
Sleep-bound	4	32	44.840	19092	1115080.45
Sleep-bound	4	64	38.490	17309	1299040.37
Sleep-bound	8	1	3060.694	61213	16336.17
Sleep-bound	8	2	681.972	27247	73316.84
Sleep-bound	8	4	308.196	23258	162234.19
Sleep-bound	8	8	262.108	32149	190761.07
Sleep-bound	8	16	102.895	29239	485930.66
Sleep-bound	8	32	23.057	12518	2168529.06
Sleep-bound	8	64	24.093	19945	2075330.34
Sleep-bound	16	1	3174.137	63482	15752.32
Sleep-bound	16	2	762.705	30446	65556.10
Sleep-bound	16	4	440.627	32474	113474.54
Sleep-bound	16	8	217.303	31267	230092.95
Sleep-bound	16	16	94.939	27137	526654.76
Sleep-bound	16	32	28.046	14726	1782814.35
Sleep-bound	16	64	17.500	13022	2857158.04

Final Conclusions:

1. What happens to the execution time as the number of goroutines increases?

CPU-Bound:

On a single core, increasing goroutines does not decrease execution time; it actually inflates the average latency per task because the scheduler must time-slice the workload, forcing tasks to wait in a queue. On multiple cores, execution time decreases linearly until the number of goroutines equals the number of physical cores.

Mutex-Bound:

Increasing goroutines drastically increases the execution time. The overhead of context switching, lock thrashing, and parking/waking blocked goroutines dominates the CPU time, slowing down the actual computation.

Sleep-Bound:

Execution time decreases significantly as goroutines increase, because the Go scheduler can effectively hide the latency of sleeping tasks by continuously swapping in ready goroutines.

2. Does increasing the number of goroutines always increase throughput?

No, increasing goroutines only increases throughput when there are idle hardware resources or when tasks are waiting on I/O.

In purely computational tasks limited to one core, adding goroutines slightly decreases throughput due to context-switching overhead.

In highly contended scenarios (Mutex locks), adding goroutines destroys throughput, as the system spends more time managing access to the lock than executing the critical section.

3. What is the difference in program behavior between GOMAXPROCS=1 and GOMAXPROCS=NumCPU?

GOMAXPROCS=1 (Concurrency without Parallelism): Forces the Go runtime to execute all goroutines sequentially on a single operating system thread. This eliminates hardware-level race conditions and cache invalidation issues, making heavily contended tasks (like a Mutex lock) run surprisingly fast, but limits computational scaling.

GOMAXPROCS=NumCPU (True Parallelism): Spreads goroutines across all physical CPU cores. This provides massive speedups for parallelizable, independent computations (CPU-bound) and I/O tasks. However, it exposes shared states to severe hardware-level contention, causing cores to constantly invalidate each other's caches when fighting for the same memory address.

4. In which type of workload are the effects of scheduling and context switching seen the most?

Positive Effect (Sleep-Bound): The scheduler's ability to instantly context-switch when a goroutine enters a sleep state is what allows a single core to process thousands of concurrent tasks efficiently.

Negative Effect (Mutex-Bound): The scheduler is forced to rapidly switch, spin, park, and wake goroutines that are fighting over a locked resource, resulting in severe "scheduler thrashing" that degrades performance.

5. From what point does increasing concurrency become useless or even decrease performance?

Increasing concurrency(go routines) degrades performance at two distinct thresholds:

- **Hardware Saturation:** For purely independent, CPU-bound tasks, concurrency becomes useless the moment the number of goroutines exceeds the number of physical cores (GOMAXPROCS). Any additional goroutines simply add time-slicing overhead without adding processing power.
- **State Contention:** For tasks sharing memory, concurrency decreases performance the moment multiple active threads attempt to mutate the exact same state simultaneously. As the ratio of "waiting goroutines" to "active locks" increases, the system falls into lock thrashing, drastically reducing overall efficiency.

Part 3

Overview & Environment

This section details the containerization of the Go-based HTTP service using Docker and its deployment within an isolated Virtual Machine. The primary objective is to establish and verify seamless network connectivity from the Host machine to the container running inside the Guest VM.

The containerized service was explicitly deployed, executed, and tested on **VirtualBox running Ubuntu 2026** as the Guest Operating System.

Network Setup: Port Forwarding

To bridge the gap between the Host machine and the Guest VM, the VirtualBox network adapter was configured to **NAT** with **Port Forwarding**.

This specific network topology ensures that TCP traffic directed to the Host machine on port 8080 is automatically intercepted by the hypervisor, routed into the Ubuntu 2026 VM's internal network, and delivered directly to the exposed Docker container running on port 8080. This architecture allows seamless API testing from the Host using `localhost:8080`.

Execution Instructions

To build and deploy the service, the following commands must be executed within the Guest VM's terminal in the directory containing the `Dockerfile`.

Build the Docker Image:

This command packages the Go application and its required environment into a lightweight container image.

```
docker build -t ds-hwl-part3 .
```

Run the Container:

This command executes the container in the background (detached mode) and maps the exposed port to the Guest VM, preparing it to receive forwarded traffic from the Host.

```
docker run -d -p 8080:8080 --name calc-service ds-hwl-part3]
```

Observability: Logging Middleware

To ensure the service is highly observable within the containerized environment, a custom HTTP logging middleware was implemented using Go's native `log/slog` package.

This middleware intercepts all incoming HTTP traffic before it reaches the core application logic, generating structured JSON logs. These logs capture critical telemetry for every request, including:

- HTTP Method (e.g., GET)
- Request Path (e.g., `/compute`)
- Client IP Address
- Total Processing Duration

Because these logs are structured and output to standard output (`stdout`), they are automatically captured by the Docker daemon and can be efficiently monitored and queried directly from the VM environment.

Proof of Connectivity

The following visual evidence demonstrates successful communication from the Host machine to the containerized service operating inside the VirtualBox VM.

```
vboxuser@DS11: ~/test/part3
vboxuser@DS11:~/test/part3$ sudo docker build -t ds-hw1-part3 .
[*] Building 1.1s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 230B
=> [internal] load metadata for docker.io/library/golang:1.22-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/golang:1.22-alpine@sha256:1699c10032ca2582ec89a24a1312d986a3f094aed3d5c1147b19880afe40e052
=> => resolve docker.io/library/golang:1.22-alpine@sha256:1699c10032ca2582ec89a24a1312d986a3f094aed3d5c1147b19880afe40e052
=> [internal] load build context
=> => transferring context: 29B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] RUN go mod init hw1/part3
=> CACHED [4/5] COPY main.go .
=> CACHED [5/5] RUN go build -o server main.go
=> => exporting to image
=> => exporting layers
=> => exporting manifest sha256:9a5d4f34e29d02f6f7e770b300e972f6093a39230f0b746d49122abb055
=> => exporting config sha256:2140a0f1413737e39406ce3911bf4b5395dfdf3e400bb3e80e4bc0bfec001dd
=> => exporting attestation manifest sha256:b0c42630efae3661770fa9f91076057113fbf9c608dc26e160eb9c9d006f904
=> => exporting manifest list sha256:8a43f542c4bccc1795d9a608a3bf66553850bf67009644abf0eb4a67fca3ec66
=> => naming to docker.io/library/ds-hw1-part3:latest
=> => unpacking to docker.io/library/ds-hw1-part3:latest
```

Figure 4: Terminal Output - Docker Image Build: Execution of the `docker build` command within the Guest VM to package the Go HTTP service into a container image.

```
vboxuser@DS11:~/test/part3$ docker run -d -p 8080:8080 --name calc-service ds-hw1-part3
1e396cd6b40f3340138adfc777b9e4f0e2d31c4e7223bbcd30455a1bc83bc1a6
vboxuser@DS11:~/test/part3$
```

Figure 5: Terminal Output - Container Execution: Starting the Docker container in detached mode within the Guest VM and mapping port 8080 to expose the service.

```
vboxuser@DS11:~/test/part3$ curl -i "http://localhost:8080/compute?op=div&a=10&b=5"
HTTP/1.1 200 OK
Content-Type: application/json
Date: Thu, 14 May 2026 12:05:20 GMT
Content-Length: 44

{"operation":"div","a":10,"b":5,"result":2}
```

Figure 6: Terminal Output - Guest Local Test: A successful `curl` request made locally from within the Guest VM, verifying that the containerized service is responding correctly before testing Host connectivity.

```
~/UT/DistributedComputing/HW (main*) > curl -i "http://localhost:8080/compute?op=add&a=10&b=0"
HTTP/1.1 200 OK
Content-Type: application/json
Date: Thu, 14 May 2026 11:31:54 GMT
Content-Length: 45

{"operation":"add","a":10,"b":0,"result":10}
```

Figure 7: Terminal Output - Host Requesting Addition: Successful Host-to-VM connectivity demonstration. The Host machine requests the `/compute` endpoint with `op=add`, and the containerized service returns the correctly calculated JSON result.

```

~/UT/DistributedComputing/HW (main*) » curl -i "http://localhost:8080/compute?op=div&a=10&b=0"
HTTP/1.1 400 Bad Request
Content-Type: text/plain; charset=utf-8
X-Content-Type-Options: nosniff
Date: Thu, 14 May 2026 11:30:37 GMT
Content-Length: 30

{"error": "Division by zero"}

```

Figure 8: Terminal Output - Division by Zero Handling: Demonstration of the service's internal error management. A request attempting to divide by zero is intercepted, returning a structured JSON error message and the appropriate HTTP status code.

```

~/UT/DistributedComputing/HW/HW1 (main*) » curl -X POST "http://localhost:8080/compute?op=add&a=10"
{"error": "Method Not Allowed!"}

```

Figure 9: Terminal Output - Invalid HTTP Method Request: A `curl` request initiated from the Host machine using an unsupported `POST` method to test the API's routing constraints.

```

~/UT/DistributedComputing/HW/HW1 (main*) » curl -i "http://localhost:8080/compute?op=add&a=10"
HTTP/1.1 400 Bad Request
Content-Type: text/plain; charset=utf-8
X-Content-Type-Options: nosniff
Date: Thu, 14 May 2026 11:39:41 GMT
Content-Length: 32

{"error": "Required: op, a, b"}

```

Figure 10: Terminal Output - Missing Parameter Error: The service successfully validates incoming queries and returns an HTTP 400 Bad Request error when the required parameter is omitted.

```

kronos@0931:~/test/part3$ docker logs calc-service
Server starting on port :8080...
{"time": "2026-05-14T11:58:42.940541796Z", "level": "INFO", "msg": "Incoming Request", "method": "GET", "path": "/health", "remote_addr": "172.17.0.1:42322", "query": "", "duration": 6513}
{"time": "2026-05-14T11:58:52.851934942Z", "level": "INFO", "msg": "Incoming Request", "method": "GET", "path": "/compute", "remote_addr": "10.0.2.2:39568", "query": "op=add&a=10", "duration": 47357}
{"time": "2026-05-14T12:01:19.617123629Z", "level": "INFO", "msg": "Incoming Request", "method": "POST", "path": "/compute", "remote_addr": "10.0.2.2:46568", "query": "op=add&a=10", "duration": 16692}
{"time": "2026-05-14T12:02:51.763559961Z", "level": "INFO", "msg": "Incoming Request", "method": "POST", "path": "/compute", "remote_addr": "10.0.2.2:53374", "query": "op=add&a=10&b=10", "duration": 9807}
{"time": "2026-05-14T12:03:12.418710757Z", "level": "INFO", "msg": "Incoming Request", "method": "GET", "path": "/health", "remote_addr": "10.0.2.2:34228", "query": "", "duration": 1884}
{"time": "2026-05-14T12:03:26.702151327Z", "level": "INFO", "msg": "Incoming Request", "method": "GET", "path": "/compute", "remote_addr": "10.0.2.2:52190", "query": "op=div&a=10&b=0", "duration": 13022}
{"time": "2026-05-14T12:03:40.07306076Z", "level": "INFO", "msg": "Incoming Request", "method": "GET", "path": "/compute", "remote_addr": "10.0.2.2:40706", "query": "op=add&a=10&b=0", "duration": 70048}
{"time": "2026-05-14T12:03:55.596430778Z", "level": "INFO", "msg": "Incoming Request", "method": "GET", "path": "/compute", "remote_addr": "10.0.2.2:45338", "query": "op=div&a=10&b=5", "duration": 62078}

```

Figure 11: Terminal Output - Middleware Logs: Viewing the container logs via `docker logs`. This displays the structured JSON output generated by the custom logging middleware, capturing request methods, paths, and total processing durations.